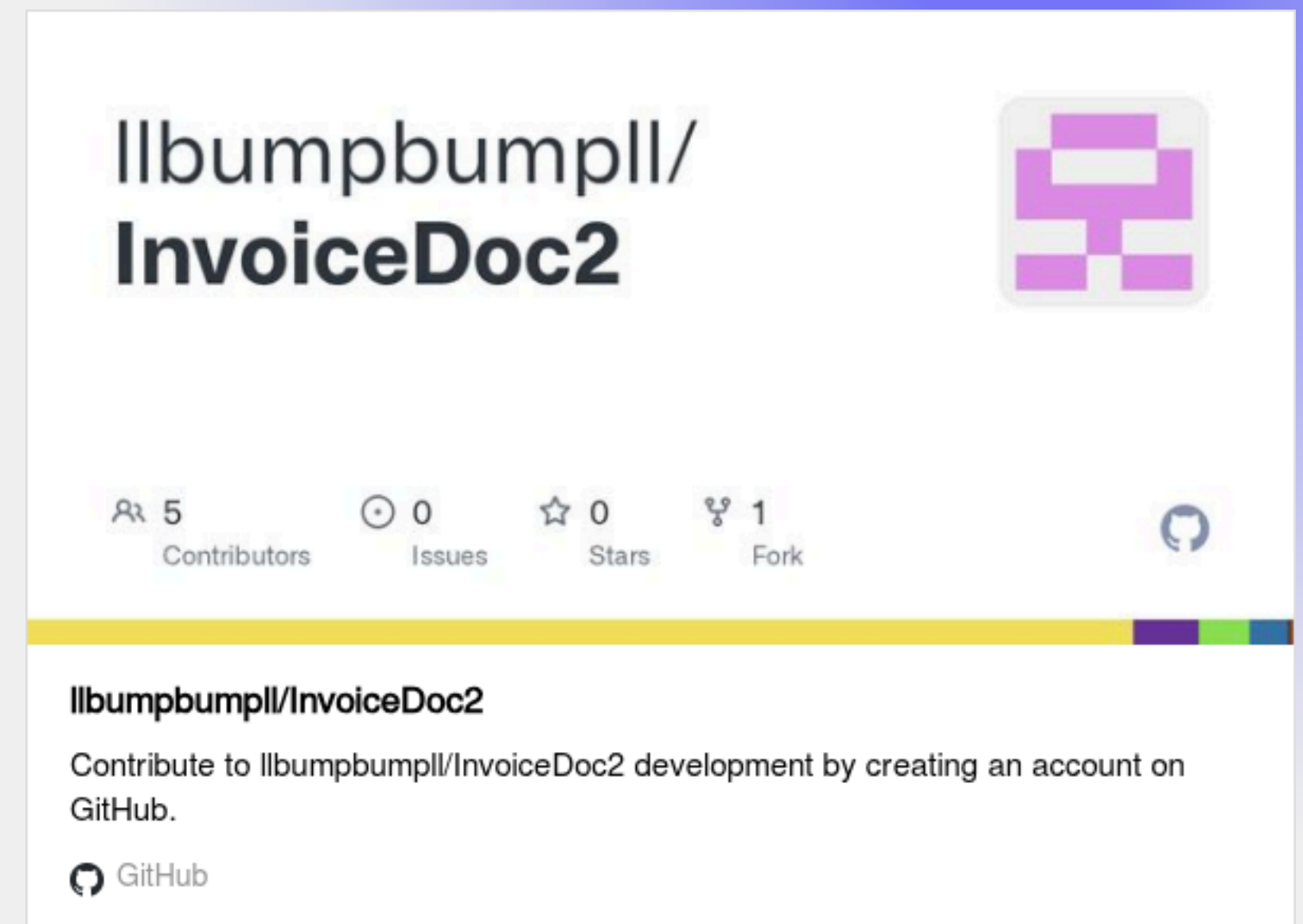


InvoiceDoc2

A full-stack Invoice Management System built with React, Express, and PostgreSQL.

GitHub Repo

GitHub



Part 1

Overview

Invoice Management System

Create, view, update, and delete

- Invoices
- Customers
- Products

Generate reports

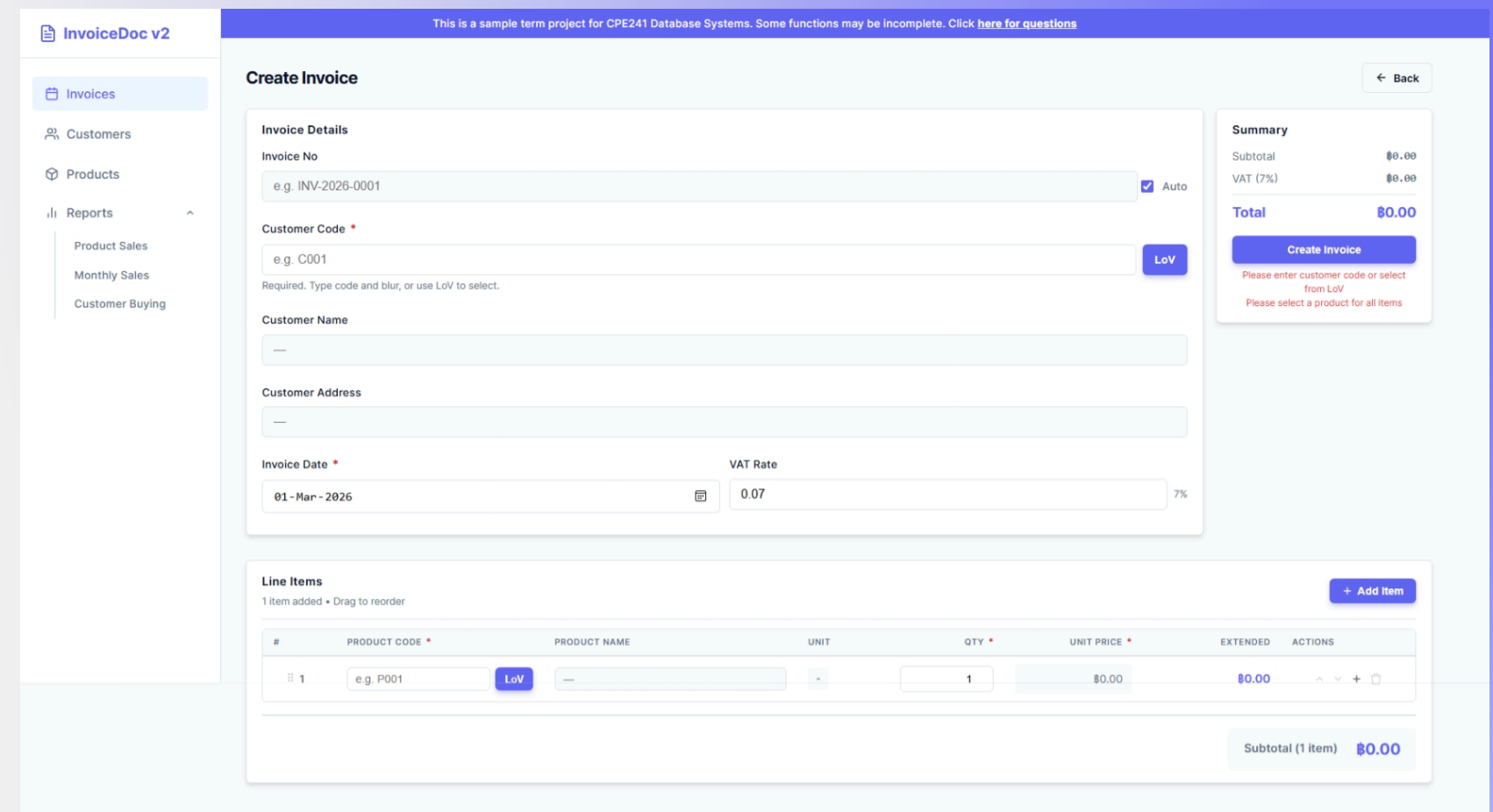
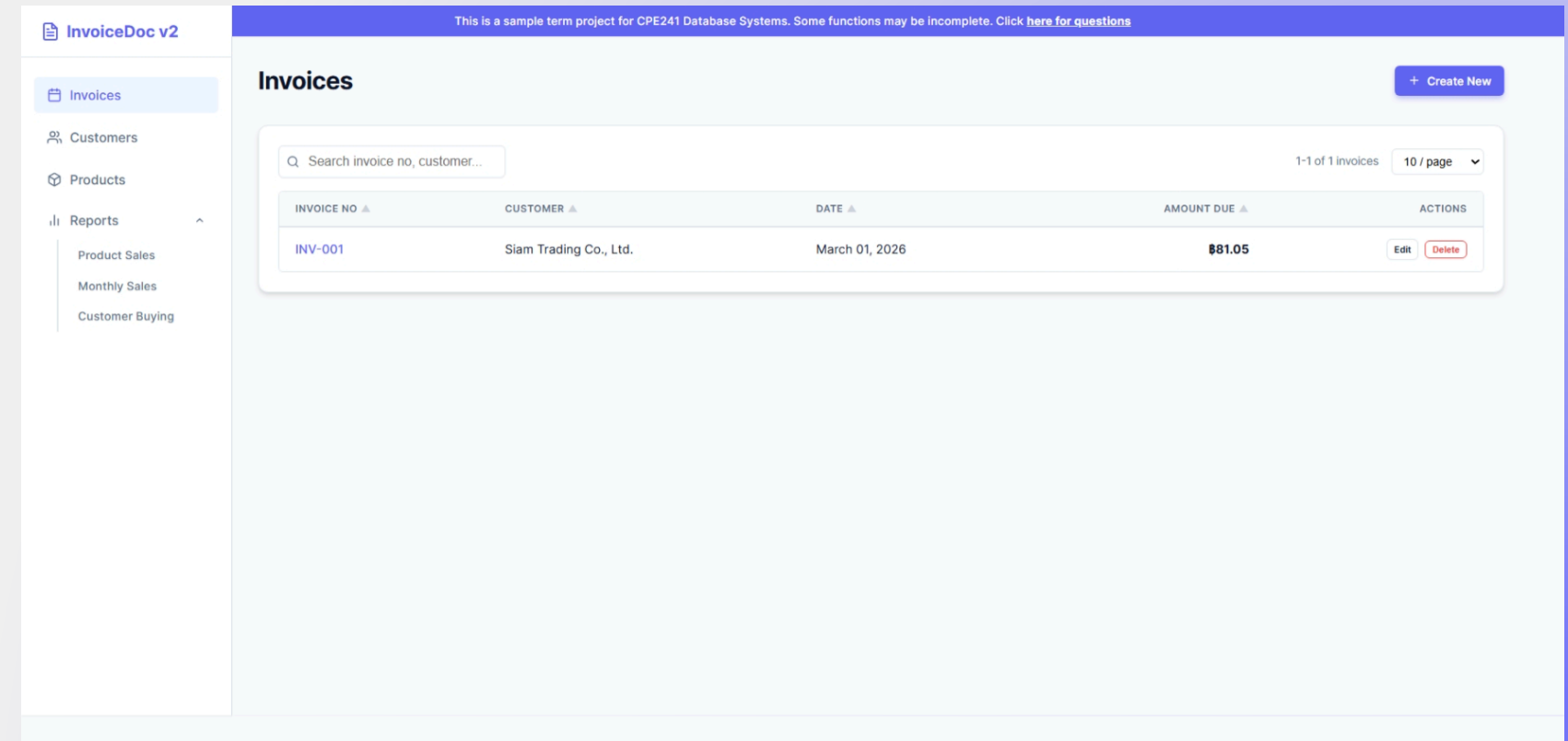
- Product sales
- Monthly sales
- Customer purchases

Invoices

View invoice list with pagination.

Create a new invoice

- Automatically generate invoice code
- Select customer
- Choose invoice date
- Apply a dynamic VAT rate
- Auto-calculate subtotal and total
- Add items and merge duplicate items

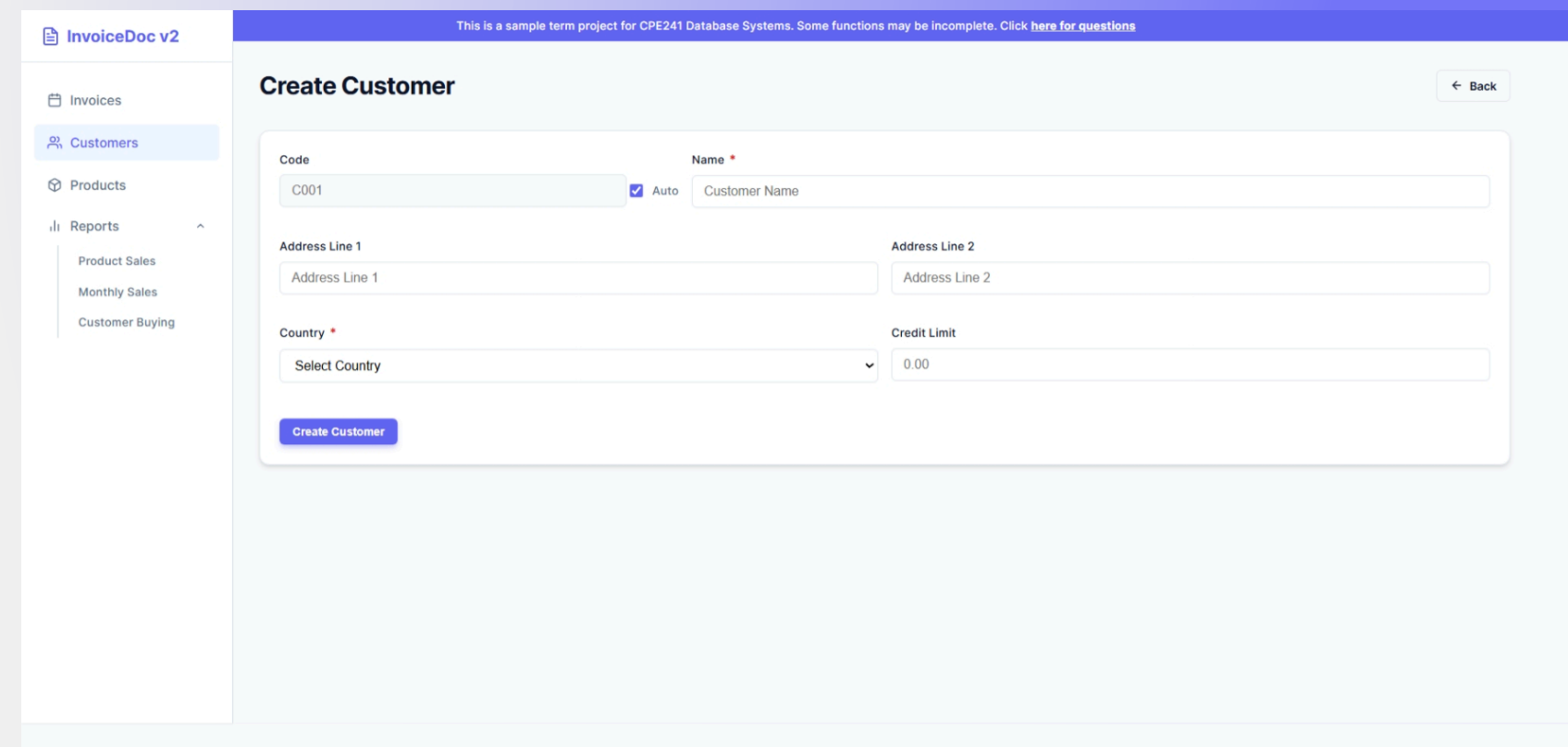
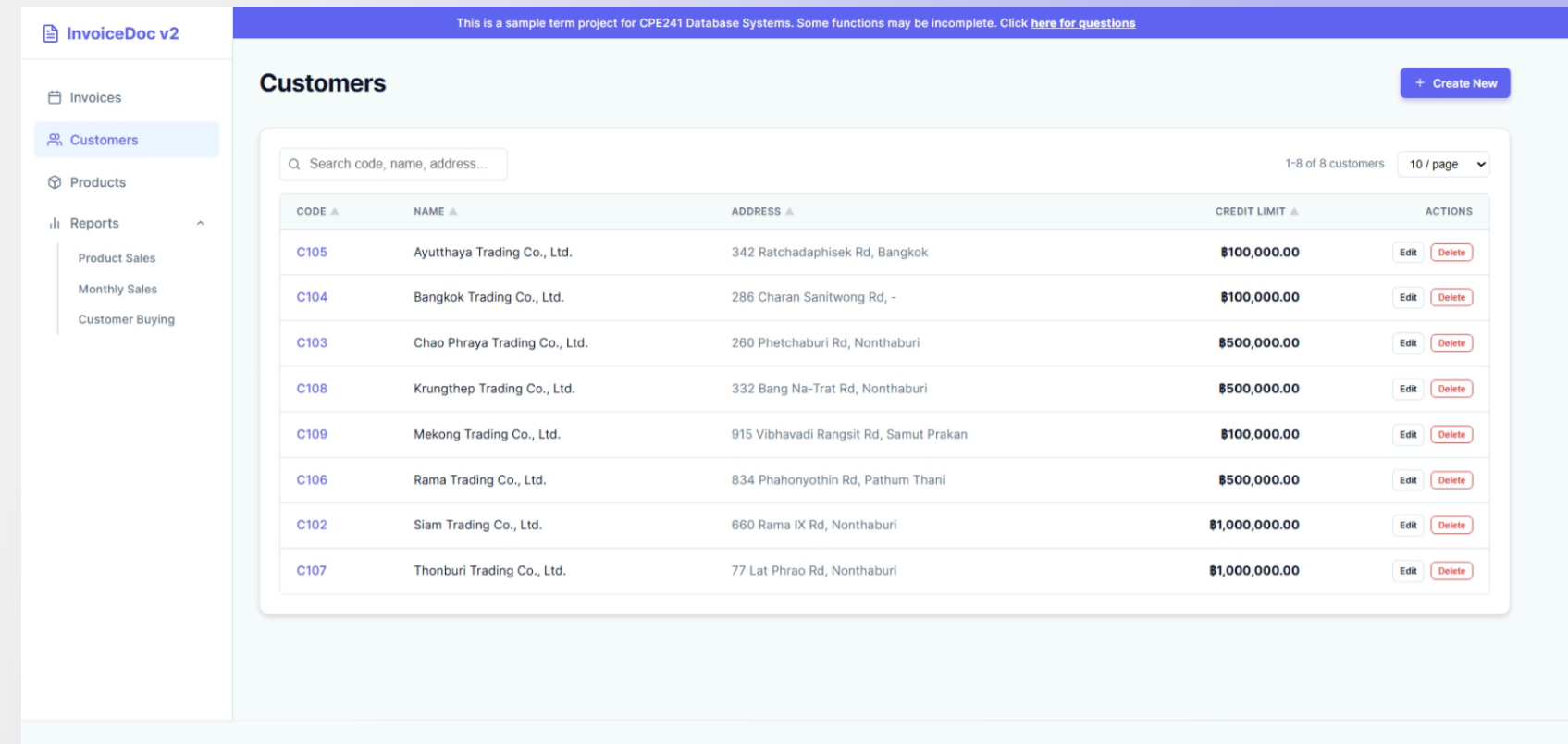


Customers

View customer list with pagination

Create a new customer

- Automatically generate customer code
- Add customer details: name, address, country, and credit limit

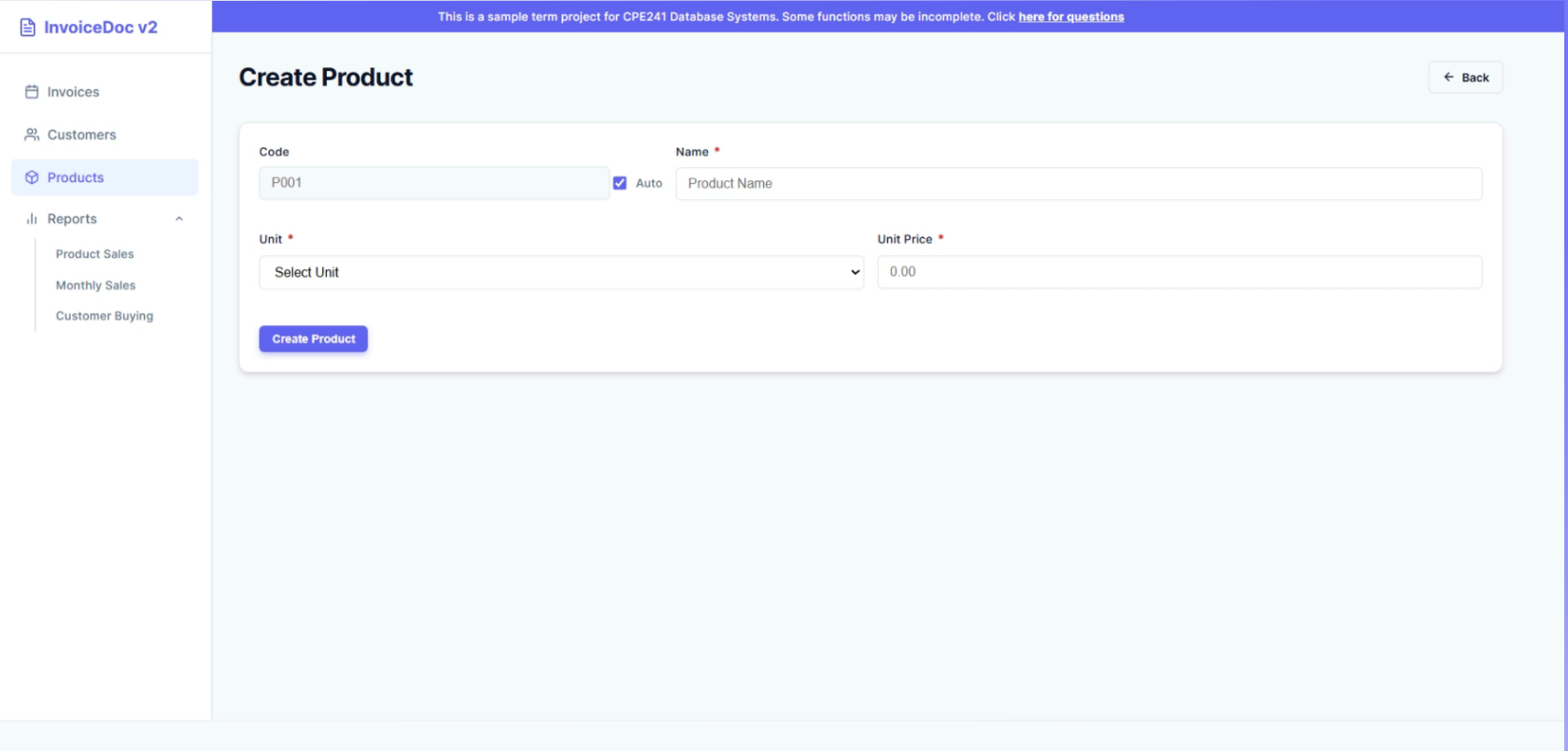
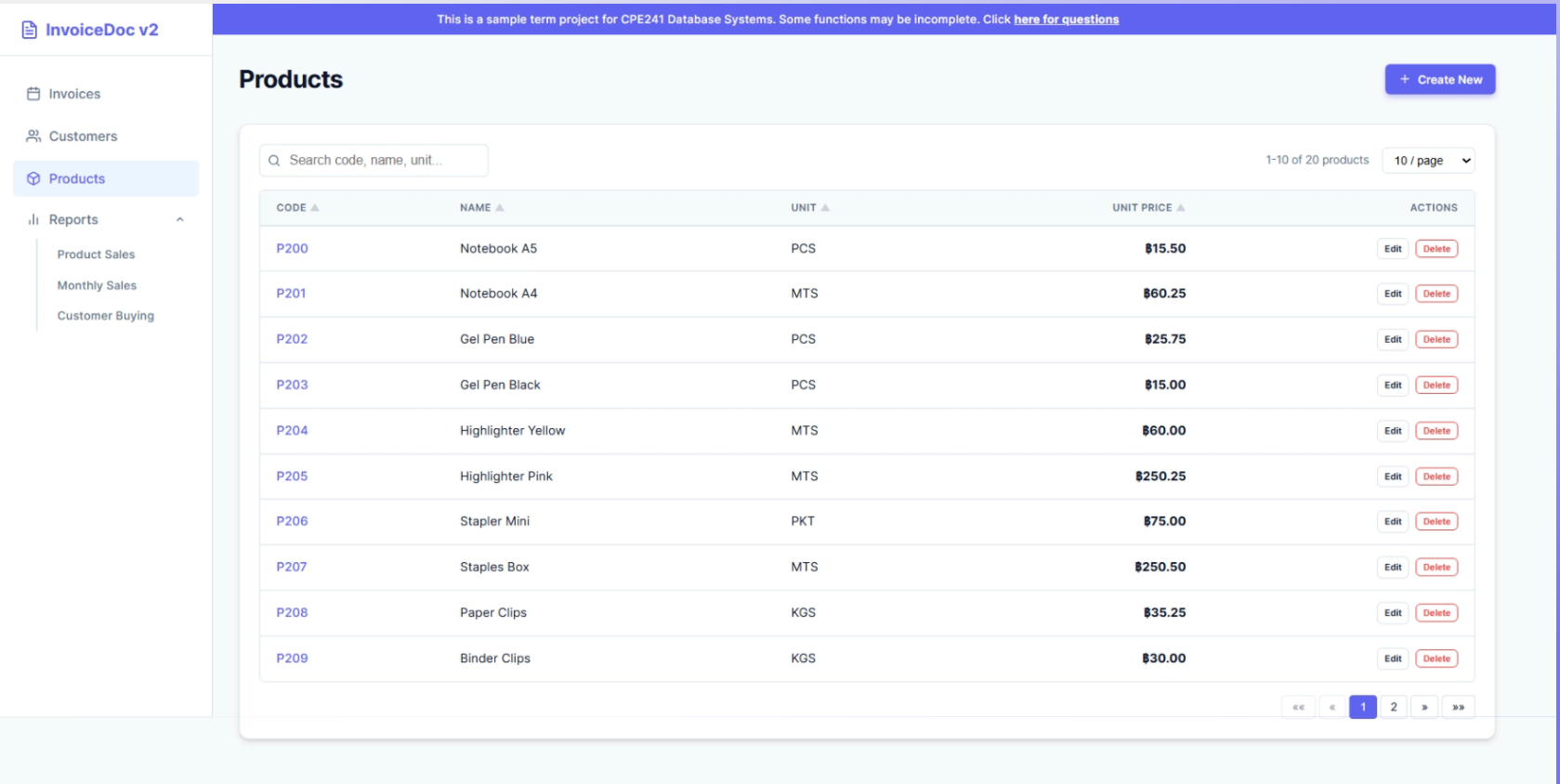


Products

View product list with pagination

Create a new product

- Automatically generate product code
- Add product details: name, unit, and price per unit



Product sales report

- Filter by product and purchase date
- Separate rows for each product
- Show the calculated total quantity sold and total revenue

InvoiceDoc v2

This is a sample term project for CPE241 Database Systems. Some functions may be incomplete. Click [here](#) for questions

Total Sales Performance
Revenue breakdown by product

Filters

Product: All Products (type code or use LoV) **LoV** Date Range: dd- - - - - yyyy to dd- - - - - yyyy **Reset** **Apply Filters**

2 total records **Refresh**

PRODUCT ▲	QUANTITY SOLD ▲	TOTAL REVENUE ▲
P200 - Notebook A5	1	\$15.50
P201 - Notebook A4	1	\$60.25

Showing 1-2 of 2 10 / page ▼

Monthly sales report

- Filter by product, buying period, and purchase date
- Separate rows by month/year and product
- Show the calculated total quantity sold and total revenue

InvoiceDoc v2

This is a sample term project for CPE241 Database Systems. Some functions may be incomplete. Click [here](#) for questions

Monthly Product Sales

Sales trends over time

Filters

Product: All Products (type code or use LoV) **LoV** Period: All Years Date Range: All Months dd- - - - - yyyy to dd- - - - - yyyy

Reset Apply Filters

2 total records Refresh

MONTH/YEAR ▲	PRODUCT ▲	QTY ▲	VALUE ▲
March 2026	P200 - Notebook A5	1	\$15.50
March 2026	P201 - Notebook A4	1	\$60.25

Showing 1-2 of 2 10 / page

Customer purchases report

- Filter by product, customer, and purchase date
- Separate rows by product and customer
- Show the calculated total quantity sold and total revenue

InvoiceDoc v2

This is a sample term project for CPE241 Database Systems. Some functions may be incomplete. Click [here](#) for questions

Customer Buying Patterns

Product purchases by customer

Filters

Product: All Products (type code or use LoV) **LoV** Customer: All Customers (type code or use LoV) **LoV** Date Range: dd- - - - - yyyy to dd- - - - - yyyy **Reset** **Apply Filters**

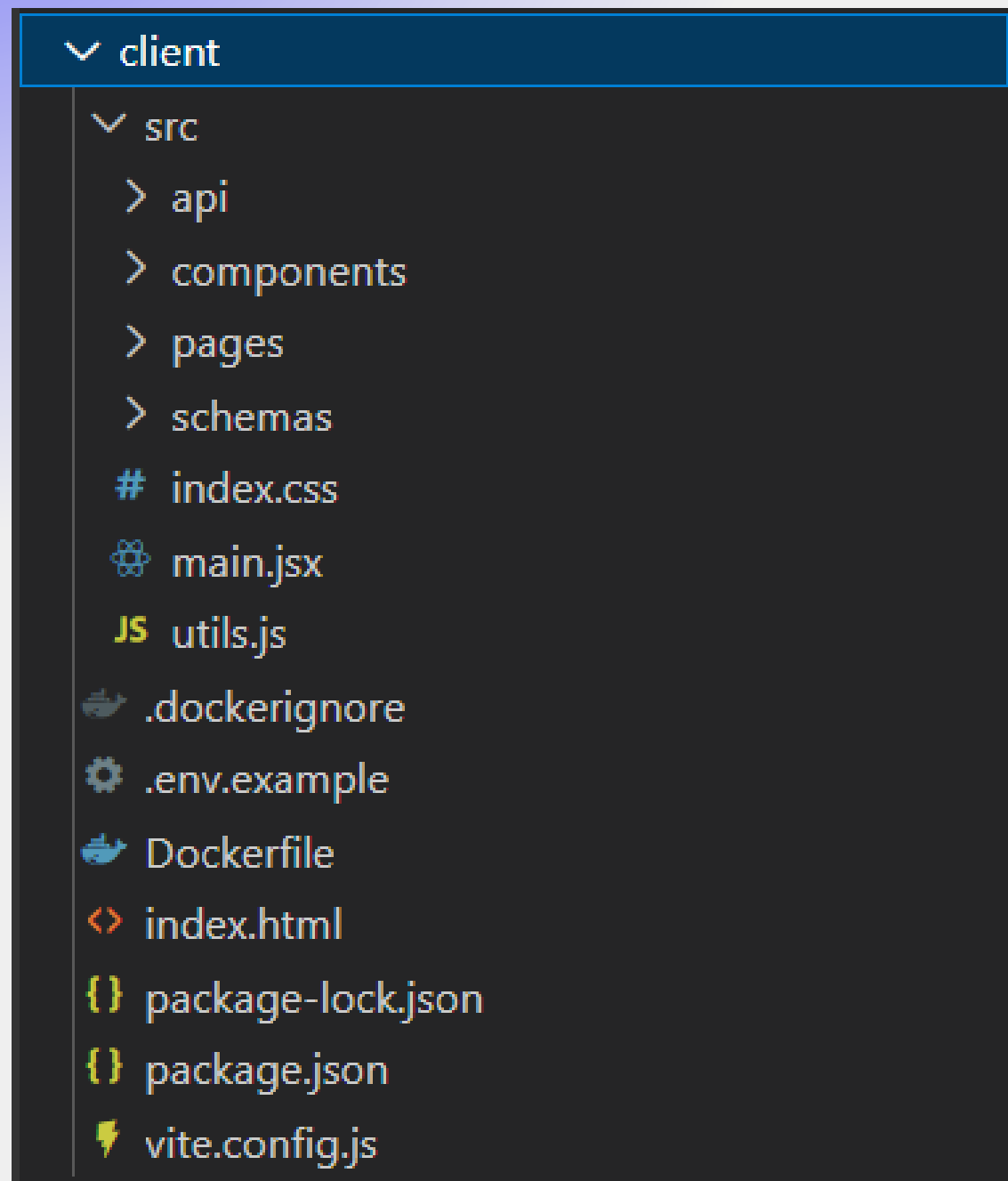
2 total records **Refresh**

PRODUCT ▲	CUSTOMER ▲	QTY ▲	VALUE ▲
P200	Siam Trading Co., Ltd. (C102)	1	\$15.50
P201	Siam Trading Co., Ltd. (C102)	1	\$60.25

Showing 1-2 of 2 10 / page ▼

File Structure

Frontend



/src/api

- Set up and call APIs in the frontend

/src/components

- Store reusable UI components

/src/pages

- Combine components, APIs, and logic to create pages

/src/schemas

- Define validation rules

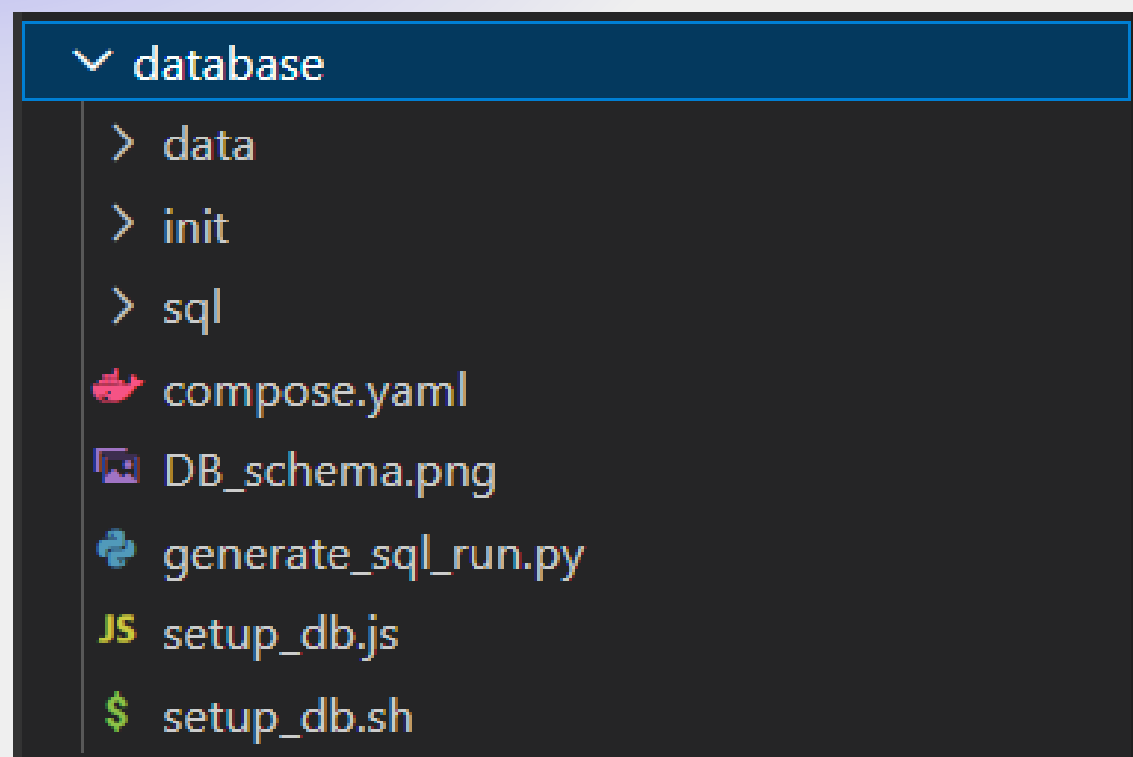
/src/utils.js

- Set format data (e.g., currency, date)

/src/main.jsx

- Main entry file of the frontend

Database



/data

- Store .csv files used as seed sources.

/init

- SQL run by Postgres container on first init.

/sql

- Main SQL scripts

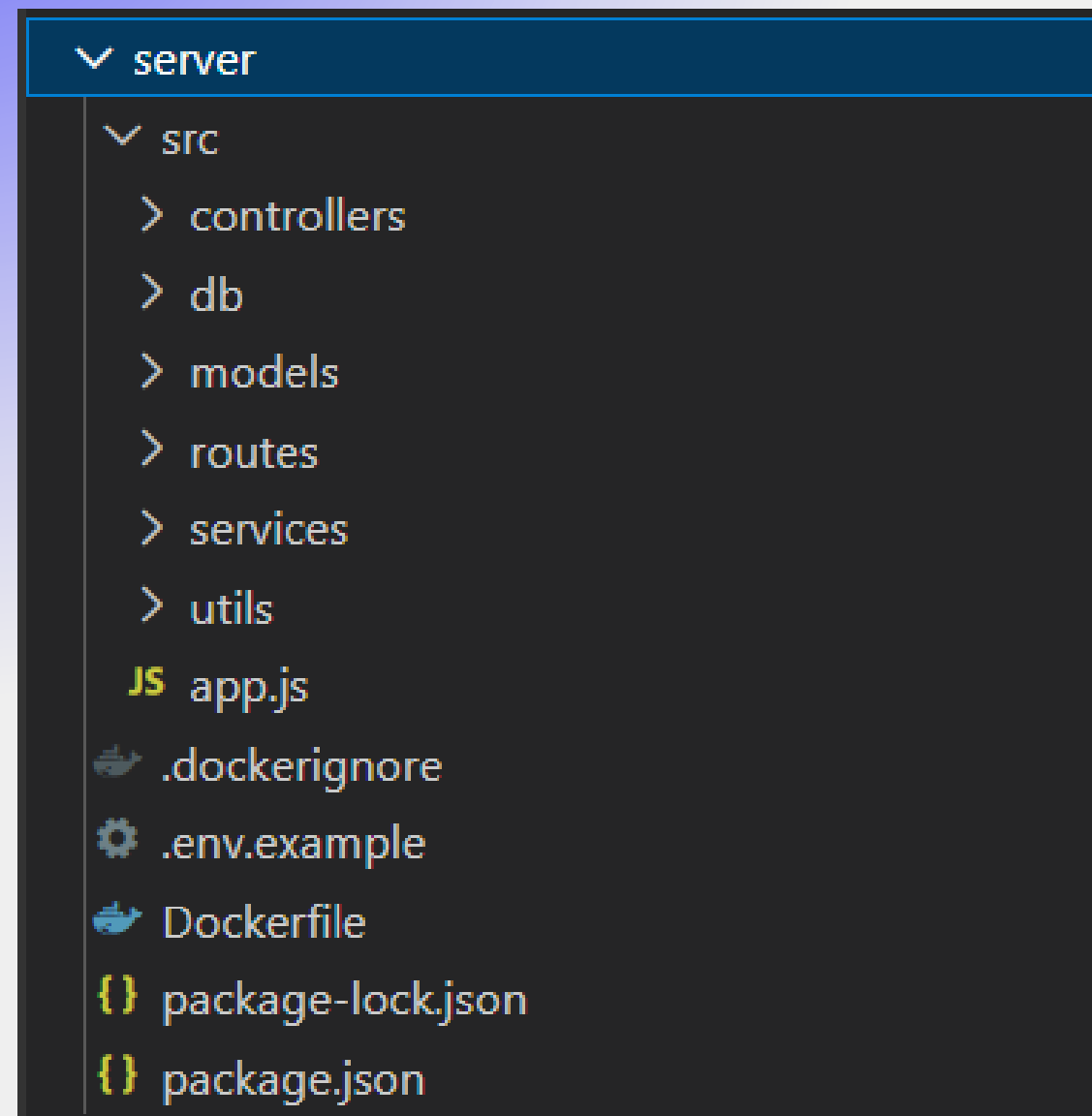
generate_sql_run.py

- Generate sql/sql_run.sql for full reset + seed.

setup_db.js / setup_db.sh

- Run database setup (apply schema, then seed when needed).

Backend



/src/app.js

- Backend entry file
- Create Express app and add middleware (CORS, JSON parsing, logging)

/src/controllers

- Handle HTTP requests and responses
- Return consistent success/error JSON

/src/db

- Set up PostgreSQL connection

/src/models

- Define Zod schemas for data validation

/src/routes

- Define API routes (URL mapping)

/src/services

- Handle business logic and SQL queries

/src/utils

- Shared helper functions (e.g., API response format, logging)